

Мультипарадигмальное программирование

Сводный отчёт по лабораторным работам №0–№5

Расширенная версия — техническая документация

Документ объединяет подробные пояснения по шести лабораторным работам курса. По каждой работе приведены: цель и постановка, теоретическая основа, структура файлов, дизайн интерфейсов, ключевые фрагменты реализации, разбор нюансов и подводных камней OpenMP, граничные случаи, юнит-тесты, инструкции по сборке (CMake и Docker), и образцы реального вывода. Для работ №4 и №5 дополнительно приведён анализ замеров производительности с конкретными числами.

Репозиторий: `mp-sequence-lab`

Платформа сборки: MSVC 14.43 (OpenMP 2.0) / g++ 13 + libomp в Ubuntu 24.04.

Содержание

Гл.	Раздел	Стр.
1	Общее устройство репозитория и сборки	2
2	Лабораторная №0. Sequence: ArraySequence / ListSequence	4
3	Лабораторная №1. Сравнение алгоритмов сортировки	7
4	Лабораторная №2. Алгоритмы поиска	10
5	Лабораторная №3. Введение в OpenMP — пять примеров	13
6	Лабораторная №4. Релаксация Якоби (OpenMP, 3D)	17
7	Лабораторная №5. Параллельное умножение матриц	20
8	Сравнительный анализ и сквозные принципы	24
9	Приложение: результаты прогонов	25

1. Общее устройство репозитория и сборки

1.1. Структура верхнего уровня

Репозиторий организован как набор из шести независимых подпроектов: lab0/ ... lab5/. У каждого своя CMake-сборка, свой Dockerfile и свой набор тестов. Такая декомпозиция облегчает сдачу: каждую работу можно собирать, тестировать и обсуждать без оглядки на соседние.

```
mp-sequence-lab/
├── README.md           # навигация по работам
├── Lab_Report.pdf       # этот документ
├── build_lab_report.py  # генератор отчёта
├── LP_0.pdf .. LP_5.pdf # отдельные методические PDF
├── lab0/ Sequence (ArraySequence, ListSequence)
├── lab1/ Sorting (Bubble, Insertion, Quick)
├── lab2/ Search (BinarySearch, BinaryTree, HashTable)
├── lab3/ OpenMP intro (5 примеров из методички)
├── lab4/ Jacobi relaxation 3D (serial vs parallel)
└── lab5/ Parallel matrix multiplication (variant №1)
```

1.2. Требования к окружению

- Компилятор с поддержкой C++17 (g++ 9+, clang++ 10+, MSVC 2019+).
- CMake 3.16 или выше — нужен для всех работ.
- OpenMP для lab3, lab4, lab5: встроен в g++ и clang++, для MSVC включается автоматически через find_package(OpenMP).
- (Опционально) Docker 20+ — у каждой лабы свой образ на Ubuntu 24.04.
- (Опционально) Ninja — ускоряет сборку через MSVC.

1.3. Универсальная сборка через CMake

В каталоге любой лабораторной:

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build --config Release
```

```
# Запуск
./build/<имя>_app      # демонстрация
./build/<имя>_tests    # юнит-тесты
```

Замечание. Для работ №4 и №5 в CMakeLists.txt явно выставлен CMAKE_BUILD_TYPE=Release, если пользователь не задал другой. Это критично для сравнения serial / parallel: без оптимизаций (-O0) различие «съедается» оверхедом OpenMP.

1.4. Docker

У каждой работы есть свой Dockerfile. Базовый образ — ubuntu:24.04; внутри ставятся git, cmake, g++, make (и libomp-dev для работ с OpenMP). Далее образ клонирует репозиторий и собирает соответствующую лабу, а CMD запускает её юнит-тесты:

```
FROM ubuntu:24.04
RUN apt-get update && apt-get install -y \
    git cmake g++ make libomp-dev \
    && rm -rf /var/lib/apt/lists/*
WORKDIR /app
```

```
RUN git clone https://github.com/MrKokao2/mp-sequence-lab.git .
WORKDIR /app/lab4
RUN cmake -S . -B build
RUN cmake --build build
CMD ["/build/jacobi_tests"]
```

Замечание. Важная особенность: Dockerfile использует `git clone` с GitHub, а не `COPY` локальных файлов. Это значит, что `docker build` собирает то, что лежит на удалённой ветке `main`. Локальные правки, не запущенные на GitHub, в контейнер не попадут.

1.5. Результаты прогона

Все шесть лабораторных собраны и протестированы и локально, и в Docker.

Лаба	Локально (MSVC 14.43)	Docker (Ubuntu 24.04)	Тесты
lab0	OK	OK	All tests passed
lab1	OK	OK	All lab1 tests passed
lab2	OK	OK	exit=0
lab3	OK	OK	All lab3 tests passed
lab4	OK	OK	All lab4 tests passed
lab5	OK	OK	All lab5 tests passed

Локальная сборка использует OpenMP 2.0 от MSVC; Docker-окружение — LLVM libomp. Оба компилятора дают идентичные результаты тестов.

2. Лабораторная работа №0. Sequence

2.1. Тема и цель

Разработка обобщённой структуры данных «последовательность» с двумя реализациями: на динамическом массиве и на двусвязном списке. Цель — освоить шаблоны C++, наследование от абстрактного интерфейса, и приём UDD (Use-Driven Design): сначала пишутся контрактные тесты, под них подстраиваются обе реализации.

2.2. Теоретическая основа

«Последовательность» — абстрактный тип, упорядоченный набор элементов с операциями вставки, удаления и доступа по индексу. В C++ STL роль такого АД выполнят контейнеры `std::vector`, `std::deque`, `std::list`. Учебная задача — написать собственный обобщённый интерфейс и две принципиально разных реализации, чтобы понять, как выбор внутреннего представления сказывается на стоимости операций.

2.3. Структура проекта

```
lab0/
├── include/
│   ├── Sequence.h          – абстрактный интерфейс
│   ├── ArraySequence.h     – реализация на std::vector
│   └── ListSequence.h      – реализация на std::list
├── src/
│   ├── main.cpp            – пустой entry-point
│   └── sequence.cpp        – заглушка для сборки SHARED-библиотеки
├── tests/test_sequence.cpp – общие тесты для обеих реализаций
├── CMakeLists.txt
└── Dockerfile
```

2.4. Дизайн интерфейса

```
template<typename TElement>
class Sequence {
public:
    virtual ~Sequence() = default;

    virtual int  getLength() const = 0;
    virtual bool getIsEmpty() const = 0;

    virtual TElement Get(int index) const = 0;
    virtual TElement GetFirst()          const = 0;
    virtual TElement GetLast()           const = 0;

    virtual void Append(TElement item)          = 0;
    virtual void Prepend(TElement item)         = 0;
    virtual void InsertAt(int index, TElement item) = 0;
    virtual void Remove(TElement item)          = 0;
};
```

Каждая реализация дополнительно предоставляет `GetSubsequence(start, end)`, возвращающую новый объект того же типа. Метод не включён в интерфейс, потому что его возвращаемый тип зависит от наследника, а виртуальные методы не умеют ковариантно возвращать значение (только указатель/ссылку).

2.5. Реализация ArraySequence

```

void Append(TElement item) override { data.push_back(item); }
void Prepend(TElement item) override { data.insert(data.begin(), item); }

void InsertAt(int index, TElement item) override {
    if (index < 0 || index > getLength())
        throw std::out_of_range("Index out of range");
    data.insert(data.begin() + index, item);
}

```

2.6. Реализация ListSequence

```

TElement Get(int index) const override {
    checkIndex(index);
    auto it = data.begin();
    std::advance(it, index); // O(index)
    return *it;
}

void Prepend(TElement item) override { data.push_front(item); } // O(1)

```

2.7. Сравнение асимптотики

Операция	ArraySequence	ListSequence	Кто выигрывает
Get(i)	O(1)	O(i)	ArraySequence
Append	амортизированно O(1)	O(1)	паритет
Prepend	O(n)	O(1)	ListSequence
InsertAt(i)	O(n) — сдвиг	O(i) — поиск + O(1) вставка	ListSequence для больших i
Remove	O(n)	O(n)	паритет
Память на 1 элемент	sizeof(T)	sizeof(T) + 2 указателя	ArraySequence
Cache-friendliness	высокая	низкая	ArraySequence

2.8. Особенности и нюансы

- Один набор тестов на две реализации. Шаблонная функция `runSequenceTest<TSequence>()` параметризуется типом и проверяет одну и ту же контрактную семантику для `ArraySequence<int>` и `ListSequence<int>`. Это даёт уверенность, что реализации взаимозаменяемы.
- Проверка границ. Все методы доступа по индексу бросают `std::out_of_range`. Это поведение явно покрыто в тестах через `try/catch`: если исключение не вылетело — `assert(false)`.
- `InsertAt` принимает индекс `== length`. Это аналогично `vector::insert(end(), x)` и означает «вставить в конец». Это разрешено намеренно, чтобы `InsertAt` был выразимее, чем простой `Append`.
- `GetSubsequence` не входит в виртуальный интерфейс. Возвращаемый тип у каждой реализации свой, а виртуальные методы не умеют ковариантно возвращать значение другого типа (ковариантность работает только для указателей/ссылок).
- Сборка как SHARED-библиотека. `CMakeLists.txt` собирает `libsequence` как SHARED. Поскольку весь код шаблонный, в `.so/.dll` попадает только маркер-символ `sequence_library_marker` — это упражнение на устройство сборки, а не на ABI шаблонов.

6. Exception safety. Все мутирующие методы либо успешно завершаются, либо бросают исключение и не оставляют контейнер в несогласованном состоянии (strong guarantee), так как `vector::insert` и `list::insert` сами имеют strong guarantee для скалярных типов.

2.9. Что проверяют тесты

- После создания: длина = 0, `isEmpty` = true.
- После Append: длина увеличивается, `GetFirst/GetLast` корректны.
- Доступ по отрицательному индексу и по индексу `>= length` бросает `out_of_range`.
- Prepend меняет первый элемент.
- `GetSubsequence` возвращает корректный фрагмент.
- Все проверки идентичны для `ArraySequence` и `ListSequence`.

2.10. Образец работы тестов

```
$ ./build/sequence_tests  
All tests passed
```

3. Лабораторная работа №1. Сравнение алгоритмов сортировки

3.1. Тема и цель

Реализовать три алгоритма сортировки разной асимптотики через общий интерфейс `ISorter<T>` и сравнить их время выполнения на одних и тех же данных. Цель — закрепить полиморфизм, освоить замеры через `std::chrono`, и на практике увидеть разницу между квадратичными и линейно-арифметическими сортировками.

3.2. Реализованные алгоритмы и баллы

Алгоритм	Балл	Среднее	Худший	Стабильность	In-place
Bubble Sort	3	$O(n^2)$	$O(n^2)$	стабильная	да
Insertion Sort	5	$O(n^2)$	$O(n^2) / O(n)$ на отсортированных данных	стабильная	да
Quick Sort	10	$O(n \log n)$	$O(n^2)$ теоретически	нестабильная	да (по результату)

Суммарно — 18 баллов.

3.3. Дизайн: общий интерфейс `ISorter`

```
template<typename T>
class ISorter {
public:
    virtual ~ISorter() = default;
    virtual ArraySequence<T> Sort(const ArraySequence<T>& sequence) = 0;
    virtual std::string GetName() const = 0;
};
```

`Sort` принимает входную последовательность по `const`-ссылке и возвращает новую отсортированную копию. Это упрощает бенчмарки: каждый запуск получает свежий исходник, ничего не «остывает» по сравнению с предыдущим прогоном.

3.4. Фрагменты реализации

Bubble Sort — без раннего выхода:

```
for (int i = 0; i < result.getLength() - 1; ++i) {
    for (int j = 0; j < result.getLength() - i - 1; ++j) {
        if (result.Get(j) > result.Get(j + 1)) {
            T temp = result.Get(j);
            result.Set(j, result.Get(j + 1));
            result.Set(j + 1, temp);
        }
    }
}
```

Insertion Sort:

```
for (int i = 1; i < result.getLength(); ++i) {
    T key = result.Get(i);
    int j = i - 1;
    while (j >= 0 && result.Get(j) > key) {
        result.Set(j + 1, result.Get(j));
        --j;
    }
}
```

```
    result.Set(j + 1, key);
}
```

Quick Sort (схема Хоара):

```
T pivot = sequence.Get((left + right) / 2);
int i = left, j = right;
while (i <= j) {
    while (sequence.Get(i) < pivot) ++i;
    while (sequence.Get(j) > pivot) --j;
    if (i <= j) {
        std::swap(sequence[i], sequence[j]); // через Set/Get
        ++i; --j;
    }
}
if (left < j) quickSort(sequence, left, j);
if (i < right) quickSort(sequence, i, right);
```

3.5. Особенности и нюансы

1. Полиморфизм через `ISorter<T>`. Шаблонная функция `MeasureSortTime` принимает `ISorter<T>&`; добавление нового алгоритма — это просто новый класс-наследник, не нужно трогать бенчмарк.
2. Bubble Sort без оптимизации. Намеренно нет раннего выхода при отсутствии обменов. Это даёт честный верхний край асимптотики $O(n^2)$ и заметную разницу с QuickSort на больших размерах. При желании в одну строку добавляется флаг `swapped`.
3. Quick Sort с pivot в середине. Выбор `pivot = (left + right) / 2` делает алгоритм устойчивым к патологическим входам: и на уже отсортированных, и на обратно отсортированных данных получается $O(n \log n)$. С классическим выбором `pivot = arr[left]` на этих же входах был бы $O(n^2)$.
4. Схема Хоара, не Ломуто. Реализация использует двусторонний проход (`i++/j--`) — это даёт меньше обменов, чем Ломуто, особенно на массивах с дубликатами.
5. Insertion Sort адаптивен. На почти отсортированных данных внутренний `while` почти не выполняется $\rightarrow O(n)$. Это важное свойство, которым пользуются гибриды (Timsort, introsort): на маленьких подмассивах вызывается именно вставочная сортировка.
6. Замер через `std::chrono::high_resolution_clock`. На Windows это `QueryPerformanceCounter` с разрешением ~ 100 нс. Точность измерения — миллисекунды; для малых n (< 1000) Bubble/Insertion могут показать 0 ms — это норма.
7. Источник случайности — `std::mt19937`. Генератор инициализируется из `std::random_device`; каждый запуск приложения даёт другую последовательность. Для воспроизводимых бенчмарков можно зафиксировать `seed`.
8. Стабильность. Bubble и Insertion — стабильные (равные элементы сохраняют относительный порядок). QuickSort в данной реализации — нестабильный (обмены через `swap` могут перепутать позиции равных элементов).

3.6. Возможности приложения

- ручной ввод последовательности;
- генерация случайной последовательности заданной длины;
- запуск каждой сортировки по отдельности;

- режим «Compare all» — печатает время трёх алгоритмов на одних данных;
- печать текущей последовательности;
- автотесты в отдельном бинарнике.

3.7. Что проверяют тесты

- пустая последовательность остаётся пустой;
- один элемент возвращается как есть;
- случайная последовательность сортируется (проверка через `IsSorted`);
- уже отсортированная — остаётся отсортированной;
- обратно отсортированная — становится отсортированной;
- последовательность с дубликатами обрабатывается корректно;
- доступ за границы бросает `std::out_of_range`.

3.8. Образец работы

```
$ ./build/sort_tests
All lab1 tests passed
$ ./build/sort_app      # интерактивно: сгенерировать N=10000,
                        # выбрать «Compare all»
Bubble Sort:      245 ms
Insertion Sort:   89 ms
Quick Sort:       3 ms
```

4. Лабораторная работа №2. Алгоритмы поиска

4.1. Тема и цель

Сравнение трёх принципиально разных подходов к ассоциативному доступу: бинарный поиск в отсортированном массиве, бинарное дерево поиска (BST) и хэш-таблица с разрешением коллизий методом цепочек. Демонстрация — поиск студента по возрасту в одной и той же коллекции тремя способами.

4.2. Структура проекта

```
lab2/
├── include/
│   ├── Student.h          - POD: name, age, group
│   ├── BinarySearch.h     - шаблонный бин-поиск с функтором-ключом
│   ├── BinaryTree.h       - BST с операциями Insert/Find
│   └── HashTable.h        - chaining hash table
├── src/main.cpp            - демо
├── tests/test_search.cpp
├── CMakeLists.txt
└── Dockerfile
```

4.3. Бинарный поиск

```
template<typename T, typename Key>
int BinarySearch(const std::vector<T>& data, Key key,
                std::function<Key(const T&)> getter) {
    int left = 0, right = data.size() - 1;
    while (left <= right) {
        int mid = (left + right) / 2;
        if (getter(data[mid]) == key) return mid;
        else if (getter(data[mid]) < key) left = mid + 1;
        else right = mid - 1;
    }
    return -1;
}
```

`getter` — лямбда, извлекающая ключ из элемента. Это позволяет искать по произвольному полю объекта; в демо — по `Student::age`.

4.4. Бинарное дерево поиска

```
Node* insert(Node* node, Key key, Value value) {
    if (!node) return new Node(key, value);
    if (key < node->key) node->left = insert(node->left, key, value);
    else node->right = insert(node->right, key, value);
    return node;
}

Value find(Node* node, Key key) {
    if (!node) return -1;
    if (key == node->key) return node->value;
    return key < node->key ? find(node->left, key)
        : find(node->right, key);
}
```

4.5. Хэш-таблица

```
std::vector<std::list<std::pair<Key, Value>>> table;
```

```

int capacity;

int hash(Key key) { return std::hash<Key>{}(key) % capacity; }

void Insert(Key key, Value value) {
    table[hash(key)].push_back({key, value});
}

Value Find(Key key) {
    for (auto& [k, v] : table[hash(key)])
        if (k == key) return v;
    return -1;
}

```

4.6. Сравнение асимптотики

Структура	Insert (средн.)	Find (средн.)	Find (худш.)	Память
Sorted array + BinarySearch	$O(n \log n)$ — на сортировку	$O(\log n)$	$O(\log n)$	$n \cdot \text{sizeof}(T)$
Binary Tree (несбал.)	$O(\log n)$	$O(\log n)$	$O(n)$ — вырождение	$n \cdot (\text{sizeof}(T) + 2 \text{ указателя})$
HashTable (chaining)	$O(1)$	$O(1)$	$O(n)$ — все в одну цепочку	$n \cdot \text{sizeof}(\text{list})$

4.7. Особенности и нюансы

1. Поиск по полю объекта. BinarySearch получает лямбду-getter, что делает его гораздо более универсальным, чем встроенный `std::binary_search` с компаратором. Минус: `std::function` «коробит» лямбду, добавляя один indirect call на каждое сравнение. На больших объёмах лучше делать `template<typename Getter>` и принимать функтор по значению.
2. Сортировка обязательна перед бинарным поиском. В демо `main.cpp` массив студентов сначала сортируется лямбдой по возрасту, и только потом запускается поиск. Без этого шага результат не определён.
3. BST несбалансированное. На вставке уже отсортированной последовательности дерево вырождается в связный список $\rightarrow O(n)$. Для продакшена нужно AVL или красно-чёрное дерево (как `std::map`).
4. HashTable: `capacity = 100` фиксировано. Нет автоматического rehash при росте load factor — если вставить 10000 ключей, цепочки разрастутся до средней длины ~ 100 , и Find деградирует в $O(n/\text{cap})$. Также 100 — не простое число; для не-uniform ключей это может увеличить коллизии (лучше брать prime, например 101).
5. Качество `std::hash`. Для `int` `std::hash` в `libstdc++` — это identity (возвращает само число); в `MSVC` — FNV. В обоих случаях для последовательных целых modulo по неудачному `capacity` даёт предсказуемое распределение.
6. Возврат «не найдено» через -1. В BinaryTree и HashTable это закодировано буквально значением -1 — допустимо для `Value = int`, но лжёт, если -1 — валидное значение. Для общего случая стоит использовать `std::optional<Value>`.
7. Дубликаты ключей. В BST одинаковые ключи уходят в правое поддереву; в HashTable — кладутся в один и тот же список. Find вернёт первое попадание. Это намеренное поведение, удобное для демонстрации устройства коллизий.
8. Замер времени ведётся только для бинарного поиска (`std::chrono::microseconds`). Корректное сравнение всех трёх структур имеет смысл при n от нескольких тысяч; на

четырёх студентах оверхед таймера превышает сам поиск.

4.8. Что проверяют тесты

- BinarySearch находит существующий элемент по позиции.
- BinarySearch возвращает -1 для отсутствующего элемента.

4.9. Образец работы

```
$ ./build/search_app  
Binary search index: 2  
Time: 0 us  
Tree search: 2  
Hash search: 2
```

5. Лабораторная работа №3. Введение в OpenMP

5.1. Тема и цель

Знакомство с базовыми директивами OpenMP на пяти простых примерах из методички. Все примеры объединены в один интерактивный стенд `openmp_intro_app`. Цель — закрепить понимание форк-джойн модели, областей видимости переменных (`shared/private`), редукции и критических секций.

5.2. Краткая теория OpenMP

OpenMP — стандарт прагма-аннотаций (`#pragma omp ...`) для распараллеливания C/C++/Fortran-кода с разделяемой памятью. Базовая модель — `fork-join`: при входе в `#pragma omp parallel` главный поток порождает команду рабочих потоков; внутри блока они работают одновременно; при выходе из блока — синхронизация и продолжение только главного потока.

Ключевые понятия:

- `shared` — переменная общая для всех потоков (доступ к ней может вызвать гонку).
- `private` — у каждого потока своя копия, без инициализации.
- `firstprivate` — приватная копия инициализирована из исходной.
- `reduction(op:var)` — приватная копия + автоматическое комбинирование в конце региона.
- `critical` — взаимное исключение для одного блока кода.
- `barrier` — точка синхронизации (неявная на закрытии `parallel` и `for`).
- `nowait` — снять неявный барьер у конкретного `for/sections`.

5.3. Сводная таблица примеров

№	Файл	Демонстрируемые директивы
1.1	<code>example1_hello.cpp</code>	<code>#pragma omp parallel</code>
1.2	<code>example2_dotproduct_for.cpp</code>	<code>parallel for + reduction(+:sum)</code>
1.3	<code>example3_dotproduct_subroutine.cpp</code>	вложенные <code>parallel + for reduction</code> в подпрограмме
1.4	<code>example4_dotproduct_sections.cpp</code>	<code>sections nowait</code>
1.5	<code>example5_matrix_vector.cpp</code>	<code>for + critical</code> (для упорядоченной печати)

5.4. Пример 1.1. Hello World

```
#pragma omp parallel default(none)
{
    int rank = omp_get_thread_num();
    std::printf("Hello World from thread = %d\n", rank);
    if (rank == 0) {
        int size = omp_get_num_threads();
        std::printf("Number of threads = %d\n", size);
    }
}
```

`default(none)` заставляет явно указывать область видимости для каждой переменной — полезная страховка от случайных гонок данных.

5.5. Пример 1.2. Скалярное произведение через reduction

```
float sum = 0.0f;
#pragma omp parallel for reduction(+:sum)
for (int i = 0; i < n; ++i) sum += a[i] * b[i];
```

У каждого потока внутри редуccionного региона создаётся приватная копия `sum`, инициализированная нейтральным элементом операции (для `+` это `0`). По выходе из региона все приватные значения объединяются в общую переменную. Без `reduction` получили бы гонку данных и непредсказуемый результат.

5.6. Пример 1.3. Вложенный parallel + reduction в подпрограмме

```
float g_sum; // глобальная — намеренно, чтобы показать паттерн
std::vector<float> g_a, g_b;

float dotprod() {
    int n = g_a.size();
    int rank = omp_get_thread_num();
    #pragma omp for reduction(+:g_sum)
    for (int i = 0; i < n; ++i) {
        g_sum += g_a[i] * g_b[i];
        std::printf(" rank = %d i=%d\n", rank, i);
    }
    return g_sum;
}

// вызов
g_sum = 0.0f;
#pragma omp parallel
{ dotprod(); }
```

Иллюстрация: `#pragma omp for` внутри уже работающего параллельного региона распределяет итерации между уже существующими потоками. Глобальные `g_*` — намеренный приём из методички, чтобы показать «подпрограмму, разделяющую состояние с потоками».

5.7. Пример 1.4. sections nowait

```
#pragma omp parallel shared(a, b, c)
{
    #pragma omp sections nowait
    {
        #pragma omp section
        for (int i = 0; i < n / 2; ++i) c[i] = a[i] + b[i];

        #pragma omp section
        for (int i = n / 2; i < n; ++i) c[i] = a[i] + b[i];
    }
}
```

Каждая секция выполняется одним потоком; потоки внутри `sections` работают параллельно. `nowait` убирает неявный барьер на конце блока `sections`; синхронизация всё равно произойдёт на закрытии охватывающего `parallel`.

5.8. Пример 1.5. Матрица × вектор с critical для печати

```
#pragma omp parallel shared(A, b, c)
{
    int rank = omp_get_thread_num();
    #pragma omp for
```

```

for (int i = 0; i < m; ++i) {
    for (int j = 0; j < m; ++j) c[i] += A[i][j] * b[j];
    #pragma omp critical
    { std::printf(" rank=%d i=%d c[%d]=%.2f\n", rank, i, i, c[i]); }
}
}

```

Замечание. `critical` здесь применяется только к `printf`, а не к вычислениям. На сами значения это не влияет: каждый поток пишет только в свой `c[i]` — гонок данных нет. Без `critical` вывод от разных потоков перемешивался бы посимвольно.

5.9. Особенности и нюансы

1. `omp_set_num_threads` vs `num_threads()`-clause. `omp_set_num_threads(N)` меняет глобальное значение по умолчанию; `num_threads(N)` в директиве — только для одного `parallel`-региона. В этих примерах используется первый вариант, поскольку число потоков задаётся аргументом командной строки и не меняется внутри программы.
2. Reduction нельзя заменить ручным сложением в `shared`. Если убрать `reduction(+:sum)` и просто писать `sum += ...` в общую переменную, то результат будет варьироваться от запуска к запуску — это классическая гонка типа `read-modify-write`.
3. `nowait` безопасен только если за блоком нет зависимости. В Example 1.4 после `sections` сразу идёт `if (rank == 0) printf`, который не трогает `c[]`, — поэтому `nowait` ничего не ломает. Если бы следом шло чтение `c[]`, нужен был бы явный `barrier`.
4. `critical` именованный/безымянный. Все безымянные `critical` блокируют один общий мьютекс процесса. Если в программе много независимых `critical`, стоит давать им имена: `#pragma omp critical(printer)`.
5. `printf` в C++17 `thread-safe` для одного вызова, но порядок вызовов между потоками не определён. Поэтому в Example 1.5 видно, как `rank=0` и `rank=1` чередуются «через одну строку», даже без критической секции — это просто очередность планирования.
6. Детерминизм входов. Во всех примерах `a[i] = b[i] = i`; следовательно $\sum i \cdot i = n \cdot (n-1) \cdot (2n-1) / 6$. Для `n=100` ожидаемая сумма = 328350.0 — именно это значение и печатается тестами.

5.10. Что проверяют тесты

- Example 1.2 / 1.3: вычисленная сумма совпадает с эталоном $\sum i^2$.
- Example 1.4: поэлементная сумма `a[i]+b[i]` совпадает с эталоном.
- Example 1.5: `c[i] = ( $\sum j+1$ ) · b[i]` для матрицы со столбцами-индексами.

5.11. Образец работы (фрагмент)

```

$ ./build/openmp_intro_tests
Sum = 328350.000000
rank = 0 i=0
rank = 1 i=50
rank = 0 i=1
rank = 1 i=51
...
A[0]= 1.0 2.0 3.0 4.0 5.0 6.0 7.0 8.0 9.0 10.0  b[0]= 1.0
...
rank= 0 i= 0 c[0]=385.00
rank= 1 i= 5 c[5]=385.00

```

```
...  
All lab3 tests passed
```

6. Лабораторная работа №4. Релаксация Якоби в 3D (OpenMP)

6.1. Тема и цель

Параллелизация итерационного метода релаксации Якоби в трёхмерной кубической сетке. Сравнение последовательной и OpenMP-версий по времени работы и ускорению на разных размерах задачи.

6.2. Теоретическая основа

Метод Якоби — простой итерационный метод для решения системы линейных уравнений $A \cdot x = b$ или, эквивалентно, для решения дискретизованных эллиптических уравнений (например, уравнения Пуассона) в узлах сетки. В трёхмерном случае значение в каждой внутренней точке обновляется как среднее шести соседей по осям:

$$B[i][j][k] = (A[i-1][j][k] + A[i+1][j][k] + A[i][j-1][k] + A[i][j+1][k] + A[i][j][k-1] + A[i][j][k+1]) / 6.0;$$

Ключевое отличие Якоби от Гаусса-Зейделя: новое значение пишется в отдельный массив B , и старые значения A внутри одной итерации не меняются. Поэтому порядок обхода внутренних точек не влияет на результат, и метод тривиально параллелизуется. Гаусс-Зейдель сходится быстрее, но обновление in-place делает его зависимым от порядка обхода.

6.3. Алгоритм по шагам

1. Init. Точки на границе куба зануляются (граничные условия Дирихле); во внутренних узлах задаётся $A[i][j][k] = 4 + i + j + k$.
2. Relax. Для каждой внутренней точки вычисляется новое значение $B[i][j][k]$ по формуле выше.
3. Resid. Параллельно копируется B в A и вычисляется максимум $eps = \max |A - B|$.
4. Шаги 2-3 повторяются до достижения $itmax$ итераций или пока $eps < maxEps$ (по умолчанию $1e-8$).

6.4. Структура проекта

```
lab4/
├── include/Jacobi.h      — публичный класс Jacobi
├── src/
│   ├── jacobi.cpp       — relax/resid serial + parallel
│   └── main.cpp         — режимы serial / parallel / table
├── tests/test_jacobi.cpp
├── CMakeLists.txt
└── Dockerfile
```

6.5. Линейный layout 3D-массива

```
class Jacobi {
    int n_;
    std::vector<double> a_, b_;
    inline int Idx(int i, int j, int k) const {
        return (i * n_ + j) * n_ + k;
    }
}
```

```
};
```

Вместо `vector<vector<vector<double>>>` используется один плоский `std::vector<double>`. Это важно для производительности: данные лежат непрерывно, доступ по `k` идёт строго последовательно, что отлично предсказывается `prefetch`-механизмом процессора. Накладные расходы на «лестницу аллокаций» вектора-из-векторов отсутствуют.

6.6. Параллельная версия `relax`

```
void Jacobi::RelaxParallel() {
    int n = n_;
    #pragma omp parallel for schedule(static)
    for (int i = 1; i <= n - 2; ++i)
        for (int j = 1; j <= n - 2; ++j)
            for (int k = 1; k <= n - 2; ++k)
                b_[Idx(i,j,k)] = (a_[Idx(i-1,j,k)] + a_[Idx(i+1,j,k)] +
                                   a_[Idx(i,j-1,k)] + a_[Idx(i,j+1,k)] +
                                   a_[Idx(i,j,k-1)] + a_[Idx(i,j,k+1)]) / 6.0;
}
```

Параллелится только внешний цикл по `i`: соседи `i-1` и `i+1` читаются из `A`, запись идёт в `B` — гонок данных нет.

6.7. Ручной `reduction max` через `critical`

```
double Jacobi::ResidParallel() {
    int n = n_;
    double eps = 0.0;

    #pragma omp parallel
    {
        double localEps = 0.0;
        #pragma omp for schedule(static) nowait
        for (int i = 1; i <= n - 2; ++i)
            for (int j = 1; j <= n - 2; ++j)
                for (int k = 1; k <= n - 2; ++k) {
                    double e = std::fabs(a_[Idx(i,j,k)] - b_[Idx(i,j,k)]);
                    a_[Idx(i,j,k)] = b_[Idx(i,j,k)];
                    if (e > localEps) localEps = e;
                }
        #pragma omp critical
        { if (localEps > eps) eps = localEps; }
    }
    return eps;
}
```

Замечание. Почему не `reduction(max:eps)`? Эта форма доступна с OpenMP 3.1 (2011). MSVC до сих пор поддерживает только OpenMP 2.0 — там нет `reduction(max:)`. Паттерн «`localEps` → `critical`» работает в любой версии и эквивалентен по семантике. Стоимость — одна `critical` на поток, что для крупных `N` пренебрежимо мало.

6.8. Особенности и нюансы

1. Раздельные массивы `A` и `B` позволяют распараллеливать тривиально. При попытке делать «`in-place`» (как в Гаусса-Зейделя) обновлённое значение немедленно используется соседом — порядок имеет значение, и параллелизация требует красно-чёрной (`red-black`) разметки сетки.

2. `schedule(static)` выбран потому, что все итерации внешнего цикла имеют одинаковую трудоёмкость. Динамическое расписание добавило бы оверхед на синхронизацию очереди задач — без выигрыша.
3. False sharing маловероятен. Каждый поток пишет в свой непрерывный диапазон строк `i`; границы блоков выровнены далеко за пределами линии кеша (64 байта).
4. `ResidParallel` использует `nowait` — потоки не ждут друг друга на конце `for`, и сразу идут к `critical`. Это безопасно: каждый поток обновляет общий `eps` с локальным максимумом, и закрытие `parallel` обеспечит финальный барьер.
5. Release-сборка по умолчанию прописана в `CMakeLists.txt`: `if(NOT CMAKE_BUILD_TYPE) set(CMAKE_BUILD_TYPE Release)`. Без оптимизаций (`-O0`) разница `serial/parallel` «съедается» оверхедом OpenMP, и наблюдаемое «ускорение» становится бессмысленным.
6. `Verify()` — контрольная сумма. Используется в тестах как детектор регрессий: параллельная версия должна давать в точности ту же сумму, что и последовательная, иначе где-то гонка данных.
7. Сходимость и стационарное решение. Для данной `Init` и нулевых граничных условий стационарное решение — нулевое поле, к которому метод и сходится. `eps` монотонно убывает (видно в логах).
8. Печать каждой итерации. `printf("it=%4d eps=%g\n", it, eps)` вне параллельного региона — она выполняется главным потоком и не создаёт оверхеда параллельного вывода.

6.9. Закон Амдала и ожидаемое ускорение

$$S(p) \leq 1 / (s + (1 - s) / p)$$

Если доля последовательного кода — s , то теоретический предел ускорения на бесконечном числе ядер — $1/s$. Для нашего метода s мал (`relax` и `resid` — почти весь вычислительный объём), поэтому на 2 ядрах ускорение близко к 2, на 4 ядрах — близко к 4. На малых N (< 64) параллельная версия может оказаться медленнее последовательной — стоимость `fork/join` превышает выигрыш.

6.10. Образец работы

```
$ ./build/jacobi_app parallel 64 5 2
it= 1 eps=47.5
it= 2 eps=18.25
it= 3 eps=11.6435
it= 4 eps=9.44444
it= 5 eps=7.98765
Parallel: N=64 itmax=5 threads=2 time=0.0030s S=3721529.733775
```

6.11. Что проверяют тесты

- `Init` выдаёт ненулевую контрольную сумму (поле инициализировано).
- Одна итерация даёт положительный `eps` (что-то изменилось).
- Параллельная версия даёт ту же контрольную сумму, что и `serial` ($|\Delta| \leq 1e-9$).

7. Лабораторная работа №5. Параллельное умножение матриц

7.1. Тема и цель

Параллельное умножение двух квадратных матриц A и B (индивидуальный вариант №1). Цель — реализовать две схемы передачи числа потоков и сравнить их с последовательной версией.

7.2. Постановка задачи

1. Способ 1. Количество потоков задаётся как входной параметр программы; размерность матриц может не быть кратной числу потоков (это ключевой пункт варианта).
2. Способ 2. Количество потоков заранее неизвестно и не является параметром задачи (используется значение OpenMP по умолчанию — обычно равно числу логических ядер).

Параллельные реализации должны давать тот же результат, что и последовательная (с точностью до арифметики с плавающей точкой).

7.3. Структура проекта

```
lab5/
├── include/
│   ├── Matrix.h           – плотная матрица на std::vector<double>
│   └── MatMul.h           – три функции умножения
├── src/
│   ├── matmul.cpp         – реализации
│   └── main.cpp           – режимы serial / parallel / auto / bench
├── tests/test_matmul.cpp
├── CMakeLists.txt
└── Dockerfile
```

7.4. API

```
namespace matmul {
    Matrix MultiplySerial(const Matrix& a, const Matrix& b);
    Matrix MultiplyParallelFixedThreads(
        const Matrix& a, const Matrix& b, int threads); // способ 1
    Matrix MultiplyParallelAuto(
        const Matrix& a, const Matrix& b);               // способ 2
}
```

7.5. Реализация (способ 1)

```
Matrix MultiplyParallelFixedThreads(const Matrix& a,
                                    const Matrix& b,
                                    int threads) {
    CheckShapes(a, b);
    int n = a.Rows(), m = b.Cols(), k = a.Cols();
    Matrix c(n, m);

    #pragma omp parallel for num_threads(threads) schedule(static)
    for (int i = 0; i < n; ++i)
        for (int j = 0; j < m; ++j) {
            double sum = 0.0;
            for (int p = 0; p < k; ++p)
                sum += a.At(i, p) * b.At(p, j);
            c.At(i, j) = sum;
        }
```

```

    }
    return c;
}

```

7.6. Реализация (способ 2)

```

Matrix MultiplyParallelAuto(const Matrix& a, const Matrix& b) {
    CheckShapes(a, b);
    int n = a.Rows(), m = b.Cols(), k = a.Cols();
    Matrix c(n, m);
    #pragma omp parallel for schedule(static)
    for (int i = 0; i < n; ++i)
        for (int j = 0; j < m; ++j) {
            double sum = 0.0;
            for (int p = 0; p < k; ++p)
                sum += a.At(i, p) * b.At(p, j);
            c.At(i, j) = sum;
        }
    return c;
}

```

7.7. Особенности и нюансы

1. Row-major хранение матрицы в одном `std::vector<double>`: `A.At(i,j) = data[i*cols + j]`. Внутренний цикл `p` идёт по `A.At(i,p)` — последовательно по памяти. Это критично для скорости.
2. Странность с `B`. Доступ `B.At(p,j)` при фиксированном `j` и растущем `p` идёт с шагом `cols` — не оптимально для кеша. Это плата за наивный `ijk`-обход; перестановка циклов на `ikj` (тогда внутренний идёт по `B[p,...]` последовательно) была бы быстрее, но менее наглядна.
3. Параллелится внешний цикл по строкам результата. При `N`, не кратном числу потоков, `schedule(static)` сам корректно разрезает диапазон. Например, при `N=7` и `threads=3` потоки получают блоки `[0..2]`, `[3..5]`, `[6..6]`. Никаких ручных корректировок не требуется — это и есть выполнение требования варианта.
4. `num_threads` vs `omp_set_num_threads`. В способе 1 число потоков выставляется в клозе самой директивы (`num_threads(threads)`) — это локально для одного `parallel`-региона. В способе 2 директива не задаёт `num_threads`, и OpenMP берёт значение из `OMP_NUM_THREADS` или `omp_get_max_threads()`.
5. `schedule(static)` обоснован. Все строки `i` имеют одинаковую трудоёмкость (внутренний цикл длиной `k`), поэтому статическое расписание даёт оптимальный баланс без `overhead` `dynamic`.
6. `sum` — приватная переменная по умолчанию. Объявлена внутри цикла, поэтому у каждого потока (и каждой итерации) она своя; никаких `reduction/private` клозов не требуется.
7. Никаких гонок данных: каждый поток пишет только в свою строку `c.At(i, j)` — критические секции и атомики не нужны. `False sharing` на границах строк пренебрежим (`cols*sizeof(double)` обычно $\gg 64$ байт).
8. Воспроизводимость замеров: `FillRandom` принимает `seed` (по умолчанию 42; в `bench` — 1 и 2 для `A` и `B`), поэтому повторные прогоны работают с одинаковыми матрицами. Это позволяет сравнивать запуски на разных машинах.

9. Сравнение через `ApproxEqual`. Абсолютный допуск $1e-6$. Для матриц с большими значениями стоило бы перейти на относительный допуск, но при заполнении из $U(0, 10)$ на $N \leq 1024$ точности более чем достаточно.

7.8. Метрики производительности

Метрика	Формула	Смысл
$T_{\square}(\text{serial})$	$O(n^3)$	прямое тройное произведение
$T_{\square}(\text{parallel})$	$O(n^3 / p)$ при идеальном балансе	время на p потоках
Ускорение $S(p)$	$T_{\square} / T_{\square}$	во сколько раз быстрее
Эффективность $E(p)$	$S(p) / p$	доля идеального ускорения
Стоимость $C(p)$	$p \cdot T_{\square}$	потерянные машинные секунды

7.9. Реальные замеры (12-ядерный CPU, MSVC Release)

Команда: `./build/matmul_app bench 256`. Время в миллисекундах, S-N — ускорение относительно serial.

N	Serial	P-1	S-1	P-2	S-2	P-4	S-4	Auto
64	0.22	0.22	1.00×	0.19	1.14×	0.21	1.01×	0.61
128	1.90	1.67	1.13×	0.80	2.38×	0.57	3.32×	0.72
256	34.56	36.64	0.94×	17.94	1.93×	9.54	3.62×	6.24

Наблюдения:

- На малых N (64) ускорения нет. Оверхед `fork/join` сравним со временем вычислений; «параллельная на 1 потоке» идентична `serial`.
- На $N=128$ и $N=256$ ускорение масштабируется почти линейно до 4 потоков: 2 потока $\rightarrow \times 1.93$, 4 потока $\rightarrow \times 3.62$.
- Эффективность на 4 потоках — 90%. Это очень хорошо для наивного `gemm`; основная потеря — `strided`-доступ к B .
- Auto-режим (12 потоков) на $N=256$ даёт 6.24 ms — хуже, чем 4 потока на 9.54 ms. Причина: на матрице 256×256 работы на поток становится мало, и `synchronization overhead` начинает доминировать (`strong scaling limit`). Для эффективного использования всех 12 потоков нужна матрица $> 1024 \times 1024$.

7.10. Что проверяют тесты

- Умножение на единичную матрицу даёт исходную: $A \cdot I = A$.
- Известный пример $2 \times 3 \cdot 3 \times 2 = 2 \times 2$:
 $\{\{1, 2, 3\}, \{4, 5, 6\}\} \cdot \{\{7, 8\}, \{9, 10\}, \{11, 12\}\} = \{\{58, 64\}, \{139, 154\}\}.$
- Параллельные реализации совпадают с `serial` при размерах, не кратных числу потоков (7×7 на 3, 4 и 8 потоках) — это прямая проверка требования варианта.
- Несовпадение форм бросает `std::invalid_argument`.

7.11. Образец работы

```
$ ./build/matmul_app bench 256
N Serial(ms) P-1(ms) S-1 P-2(ms) S-2 P-4(ms) S-4 Auto(ms)
64 0.2154 0.2161 1.00 0.1888 1.14 0.2141 1.01 0.6119
128 1.8961 1.6736 1.13 0.7981 2.38 0.5710 3.32 0.7178
256 34.5642 36.6427 0.94 17.9376 1.93 9.5360 3.62 6.2443
```

8. Сравнительный анализ и сквозные принципы

8.1. Парадигмы и технологии по работам

№	Тема	Парадигмы и технологии
0	Sequence	ООП с шаблонами; абстрактный интерфейс + две реализации; контрактные тесты, общие для
1	Sorting	ООП через ISorter<T>; полиморфизм для бенчмарка; std::chrono high_resolution_clock.
2	Search	Шаблоны + std::function для бинарного поиска; рекурсивный BST; хэш-таблица с цепочками; л
3	OpenMP intro	#pragma omp parallel, parallel for, reduction(+), sections nowait, critical; реализация default(none)
4	Jacobi 3D	OpenMP: schedule(static), ручной reduction max через critical (переносимый эквивалент reduction
5	MatMul	OpenMP: num_threads в директиве vs omp_set_num_threads; schedule(static); row-major layout;

8.2. Сквозные принципы

1. Один абстрактный интерфейс — несколько реализаций. Этот приём проходит через lab0 (Sequence), lab1 (ISorter) и lab5 (три варианта умножения). Преимущество — бенчмарки и тесты пишутся один раз для всех реализаций.
2. Тесты как контракт. Везде есть отдельная цель *_tests, которая прогоняет автотесты без интерактивного ввода. Это позволяет проверять корректность из Docker / CI одной командой `docker run --rm mp-labN`.
3. Параллельность как ускорение, а не как самоцель. В lab4 и lab5 параллельные версии сверяются с последовательными по контрольной сумме / точечным проверкам. Без этой проверки само понятие «ускорение» теряет смысл (можно «ускорить» неверный результат).
4. Воспроизводимость. Все случайные входы инициализируются из фиксированного seed (FillRandom(..., seed=1u) и т.п.); это позволяет получать одинаковые числа при повторных запусках и сравнивать времена между разными машинами.
5. Release-сборка по умолчанию в lab4/lab5 — необходимая гигиена для замеров. Debug-сборка обнуляет смысл сравнения serial vs parallel.
6. Переносимость OpenMP-кода между MSVC 2.0 и LLVM/GCC. Этого удалось достичь, отказавшись от reduction(max:) в пользу паттерна «local + critical». Код собирается одинаково и в Visual Studio, и в Docker под Ubuntu.

8.3. Что осталось за кадром (возможные расширения)

- lab2: заменить -1 на std::optional<Value>, добавить автоматический rehash в HashTable.
- lab1: добавить адаптивный QuickSort + InsertionSort на малых подмассивах (как в std::sort).
- lab5: добавить блочное (tilled) умножение для лучшей cache-локальности; попробовать ikj-обход; сравнить с collapse(2).
- lab4: добавить red-black ordering для Гаусса-Зейделя — сходимость в ~2 раза быстрее при той же параллельной структуре.
- Везде: добавить CI на GitHub Actions, чтобы Dockerfile и push на main триггерили автоматический прогон всех тестов.

9. Приложение. Результаты прогонов

9.1. Локальная сборка (MSVC 14.43, Windows, 12-ядерный CPU)

```
=== lab0 : sequence_tests.exe ===
All tests passed
exit=0

=== lab1 : sort_tests.exe ===
All lab1 tests passed
exit=0

=== lab2 : search_tests.exe ===
exit=0

=== lab3 : openmp_intro_tests.exe ===
Sum = 328350.000000
...
All lab3 tests passed
exit=0

=== lab4 : jacobi_tests.exe ===
it= 1 eps=23.5
it= 1 eps=47.5
it= 2 eps=18.25
...
All lab4 tests passed
exit=0

=== lab5 : matmul_tests.exe ===
All lab5 tests passed
exit=0
```

9.2. Сборка и прогон в Docker (Ubuntu 24.04, g++ + libomp)

```
$ docker run --rm mp-lab0 → All tests passed (exit=0)
$ docker run --rm mp-lab1 → All lab1 tests passed (exit=0)
$ docker run --rm mp-lab2 → (silent) (exit=0)
$ docker run --rm mp-lab3 → ... All lab3 tests passed (exit=0)
$ docker run --rm mp-lab4 → ... All lab4 tests passed (exit=0)
$ docker run --rm mp-lab5 → All lab5 tests passed (exit=0)
```

9.3. Бенчмарк lab5 на 12-ядерном CPU

```
$ ./build/matmul_app bench 256

N Serial P-1 S-1 P-2 S-2 P-4 S-4 Auto
64 0.2154 0.2161 1.00 0.1888 1.14 0.2141 1.01 0.6119
128 1.8961 1.6736 1.13 0.7981 2.38 0.5710 3.32 0.7178
256 34.5642 36.6427 0.94 17.9376 1.93 9.5360 3.62 6.2443

$ ./build/matmul_app auto 256
Auto N=256 (default OMP threads=12) time=4.0821 ms
```

9.4. Вывод lab4 для N=64

```
$ ./build/jacobi_app parallel 64 5 2
it= 1 eps=47.5
it= 2 eps=18.25
it= 3 eps=11.6435
```

```
it= 4 eps=9.44444  
it= 5 eps=7.98765  
Parallel: N=64 itmax=5 threads=2 time=0.0030s S=3721529.733775
```